University of St.Gallen

Using Computer Simulation

for Understanding Organizations as Complex Systems

Week 3: How to implement simulation models with NetLogo?

Marcel Cahenzli

From insight to impact.

Structure

Date	Topic	Deliverables	Literature	Responsible
19.02.	Introduction to the course, theory foundations, simulation demos	-	papers 1, 2	Stephan & Marcel
26.02.	How to apply agent-based modelling?	-	book chapter 1	Marcel
05.03.	**How to implement simulation models with NetLogo?**	-	book chapter 2 (3, 4, 5)	Marcel
12.03.	Independent case work (*no input session*)	-		-
19.03.	Intermediate group presentation and feedback on case/model	Presentation		Stephan & Marcel
26.03.	How to run simulation experiments and analyse simulation data?	-	book chapter 6	Stephan
	(*break*)			
16.04.	Intermediate group presentation on implemented model and analysis	Presentation		Stephan & Marcel
23.04.	How to verify and validate simulation models?	-	book chapter 7	Stephan
30.04.	Independent case work (*no input session*)	-		-
07.05.	Final case presentation in groups and reflection of learnings	Presentation, Model, Analysis		Stephan & Marcel
14.05.	Ascension Day (*no input session*)	-		-
21.05.	Oral exams	Oral Exam		Stephan & Marcel

Paper 1: Beese, J., Haki, M. K., Aier, S., & Winter, R. (2019). Simulation-Based Research in Information Systems: Epistemic Implications and a Review of the Status Quo. Business & Information Systems Engineering, 61(4), 503-521.
Paper 2: Haki, K., Beese, J., Aier, S., & Winter, R. (2020). The Evolution of Information Systems Architecture: An Agent-Based Simulation Model. MIS Quarterly, 44(1), 155-184.
Book: Wilensky, U., & Rand, W. (2015). An Introduction to Agent-Based Modeling: Modeling Natural, Social, and Engineered Complex Systems with NetLogo. The MIT Press.

Agenda

1	Recap of Week 2	4
2	Introduction to NetLogo	8
3	Coding with NetLogo	21
4	Getting our hands dirty	32

Agenda

1	**Recap of Week 2**	**4**
2	Introduction to NetLogo	8
3	Coding with NetLogo	21
4	Getting our hands dirty	32

Models

What is a model?

Three essential characteristics (Stachowiak, 1973):

- a **representation**
- **reductionist**
- **pragmatic**

A model is (Wilensky & Rand):

- a **coarse-grained** representation of a phenomenon
- features its **essential components** and **mechanisms**
- **helps understand** the phenomenon

… and (thus) makes understanding more efficient than observing the phenomenon in reality.

Modelling

What does this mean for modelling?

Modelling is the process one goes through between (1) identifying phenomenon and (2) having a reductionist, pragmatic representation of that phenomenon.

1. Identify a phenomenon and formulate a question
2. Narrow down its scope:
   a) What is the use for your model?
   b) Which parts of the phenomenon are **essential** to your use case; and which parts are not?
3. What are essential components and mechanisms?
4. Represent the essential components and mechanisms in a way that helps understanding.

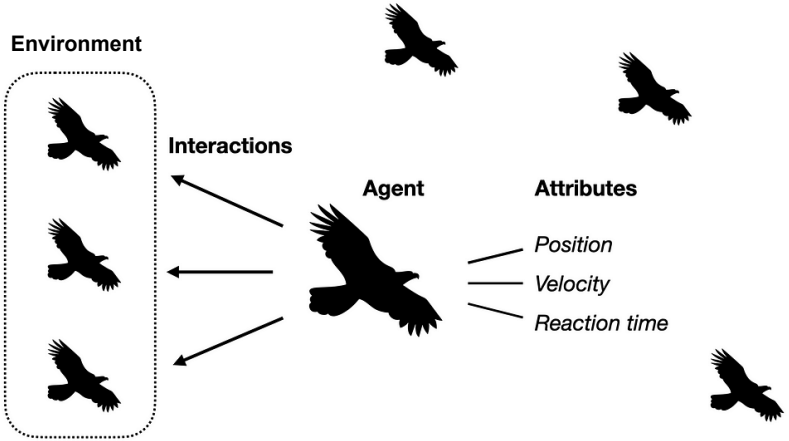
What is ABM?

Agent-based Models consist of
- **Agents** (one or several types of agents; incl. attributes)
- **Environment** (space or context for agents' actions)
- **Interactions** (update scheme for the attributes)

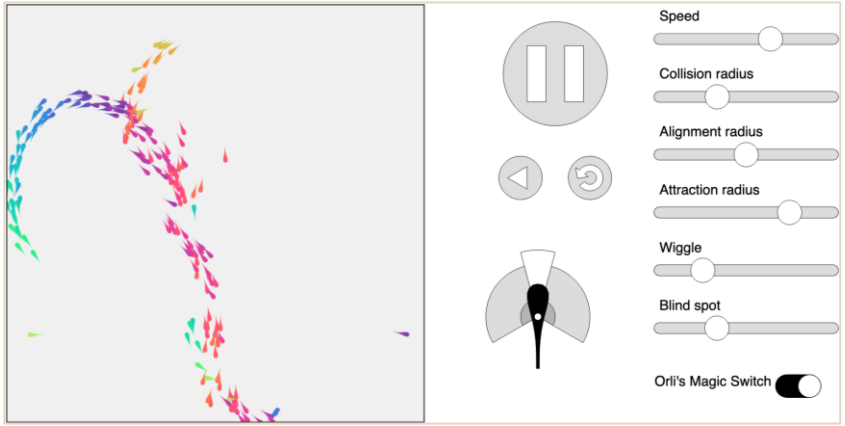
Agents are described with **relevant attributes** and fundamental behavioural rules. Interacts with agents & env.

The **environment** can be spatial or non-spatial, dynamic or static. It may thus limit or enable interaction and it may change (and thus have its own attributes) or not change over time.

Interactions consist of the interaction mechanisms (e.g., local vs. global interaction; direct vs. indirect interaction) and interaction rules (e.g., interaction protocols, strength modifiers) that govern the interactions between agents and between agents and the environment.



Source: towarddatascience.com

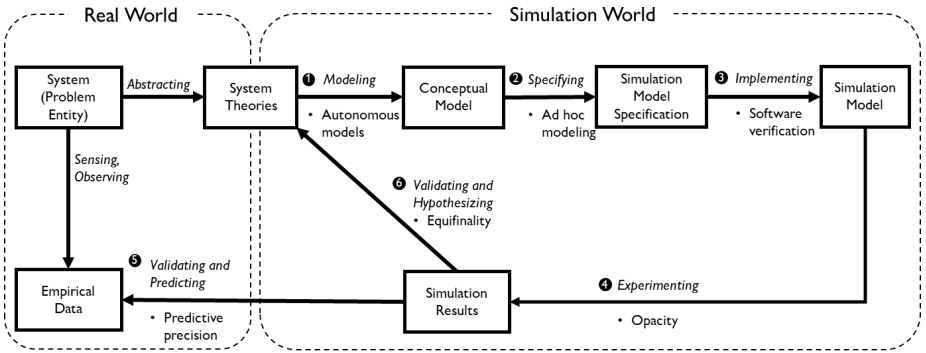


Source: complexity-explorables.org

How to apply ABM?

Approach

1. Identify a phenomenon and formulate a question
2. Narrow down the scope of the phenomenon
3. Identify and define actors (incl. relevant attributes and behaviours), environment (incl. non-spatial vs. spatial, dynamic vs. constant), & interactions (including stochasticity in actor, env., interactions).
4. **Implement the model (NetLogo, week 3)**
5. Calibrate the model
6. Run simulations (week 6)
7. Validate the model & draw conclusions (week 9)

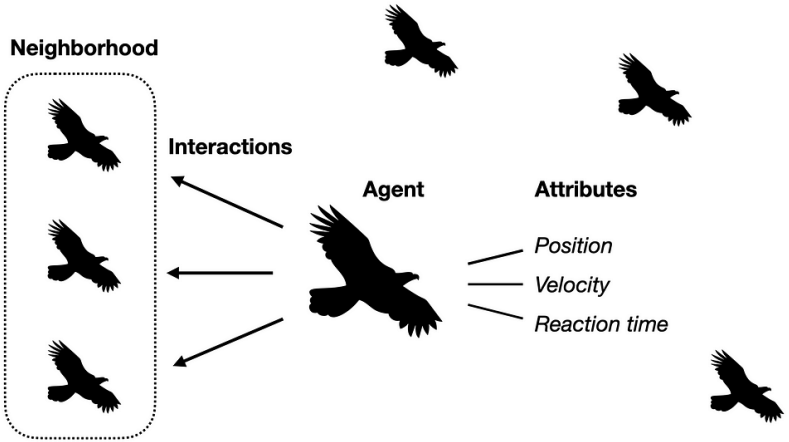


Beese, J., Haki, M. K., Aier, S., & Winter, R. (2019). Simulation-Based Research in Information Systems: Epistemic Implications and a Review of the Status Quo. Business & Information Systems Engineering, 61(4), 503-521.

Agenda

1	Recap of Week 2	4
2	**Introduction to NetLogo**	**8**
3	Coding with NetLogo	21
4	Getting our hands dirty	32

Before we start to look at code…



Source: towarddatascience.com

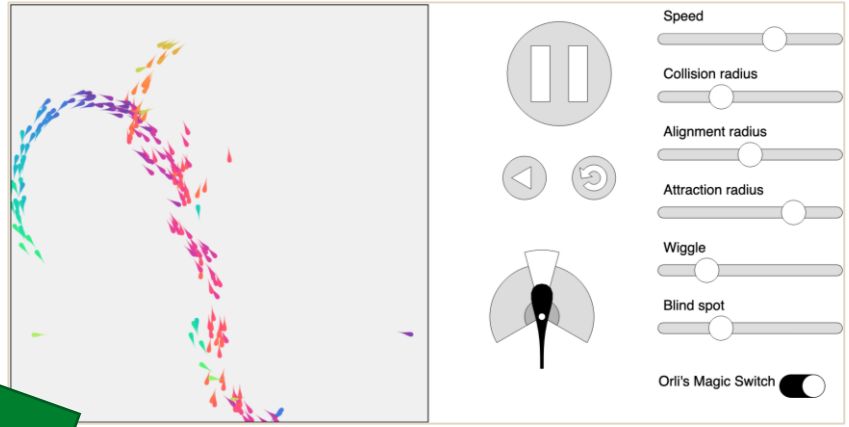
Remember: While modelling, we **must make pragmatic choices**. The selection of attributes is never complete, but it should be well-reasoned.

Start with the simplest version of your model that you can come up with (very few attributes and agent types).

Increase the level of specificity of your conceptualisation of the model to such a degree that you can 'run' the simulation on paper, including attribute values and the respective update mechanisms.

Only then, you start to code…

This is what will be reviewed in Week 5



rce: complexity-explorables.org

Introduction to NetLogo

About NetLogo and Install

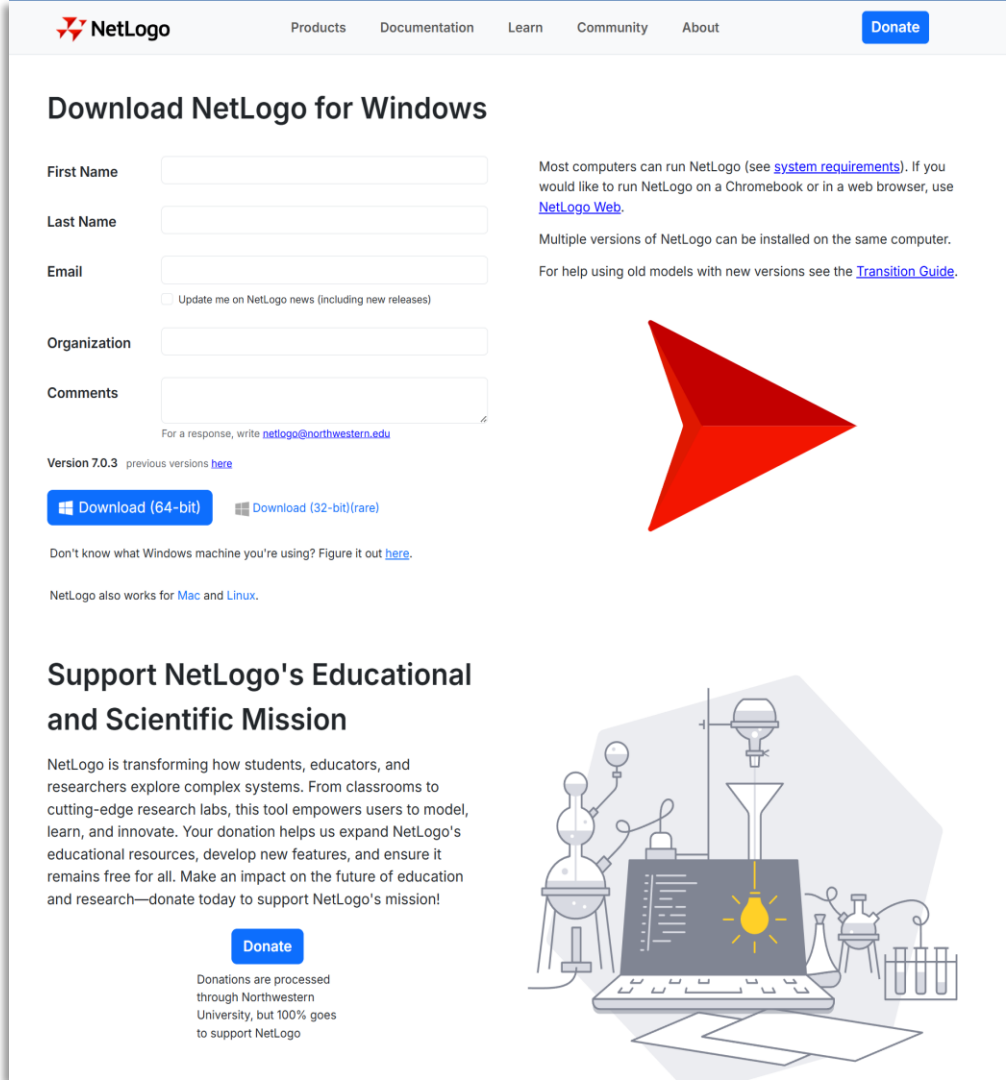
"NetLogo is a multi-agent programmable modelling environment. It is used by (…) students, teachers, and researchers worldwide. (…) It is authored by Uri Wilensky (…)." ccl.northwestern.edu

Net → reference to decentralized, interconnected nature of phenomena one may analyse with NetLogo.

Logo → reference to the Logo language, of which NetLogo is a dialect.

To install NetLogo, go to:
https://www.netlogo.org/

University of St.Gallen

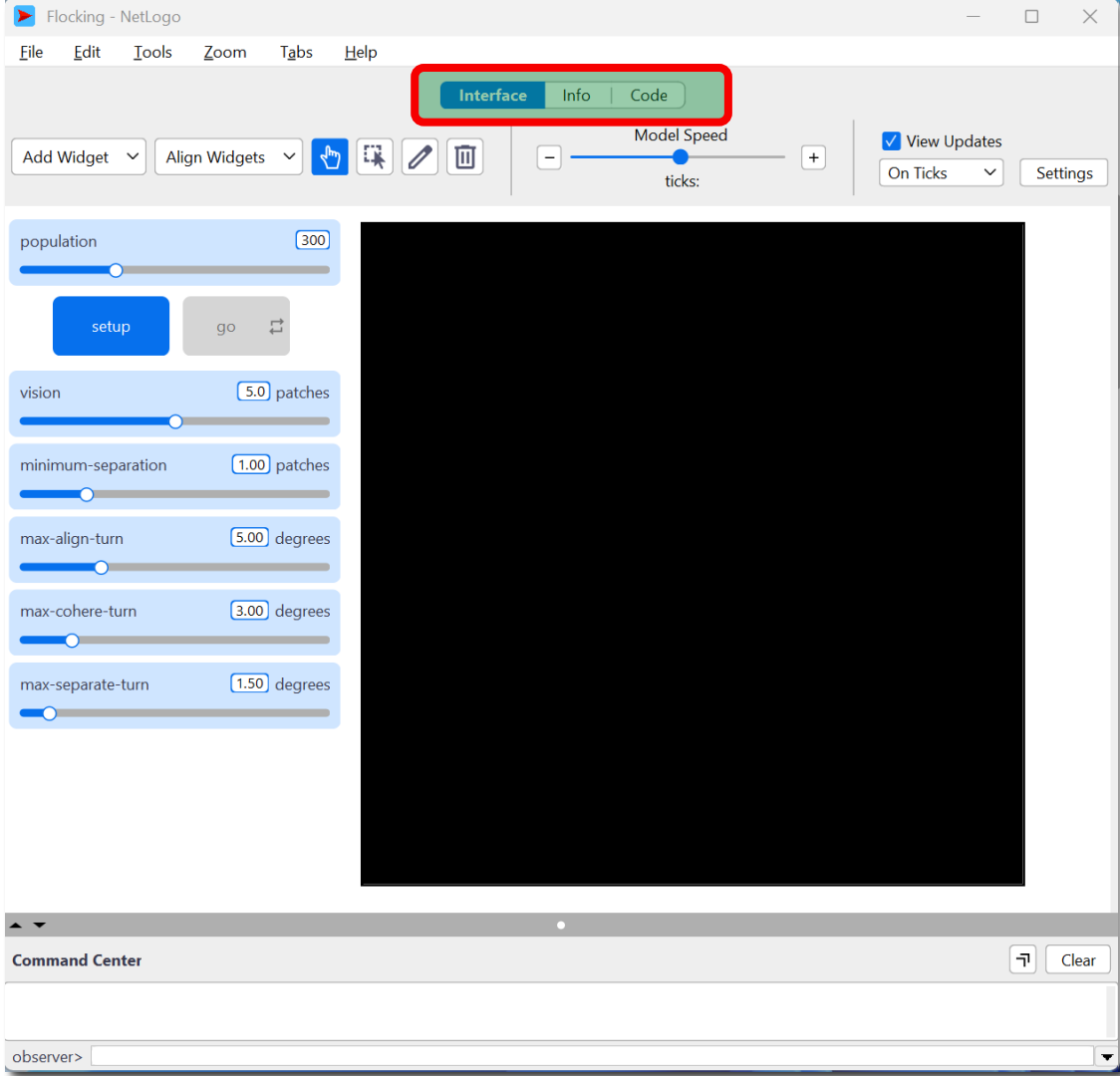


Introduction to NetLogo

Developing the right intuitions for NetLogo

3-Tab structure:

- Interface (visualisation, interaction)
- Info (description of the ABM)
- Code (logic of the ABM)



University of St.Gallen

About the Programming Part (20%)

Reminder:
Week 1

What to submit?

- NetLogo File or equivalent
  - interface (*Interface* tab)
  - explanatory description (*Info* tab)
  - documented code (*Code* tab)
  - experiment definition (Tools → *BehaviorSpace*)
- documented scripts for data analysis (e.g., in Python, Excel)
- data set (data generated through simulation experiments) on which the analysis (presentation) is based

Criteria

- needs to run without errors
- understandable as a standalone documentation

PS: If any significant changes are made after the presentation, a change summary needs to be added to the info tab.

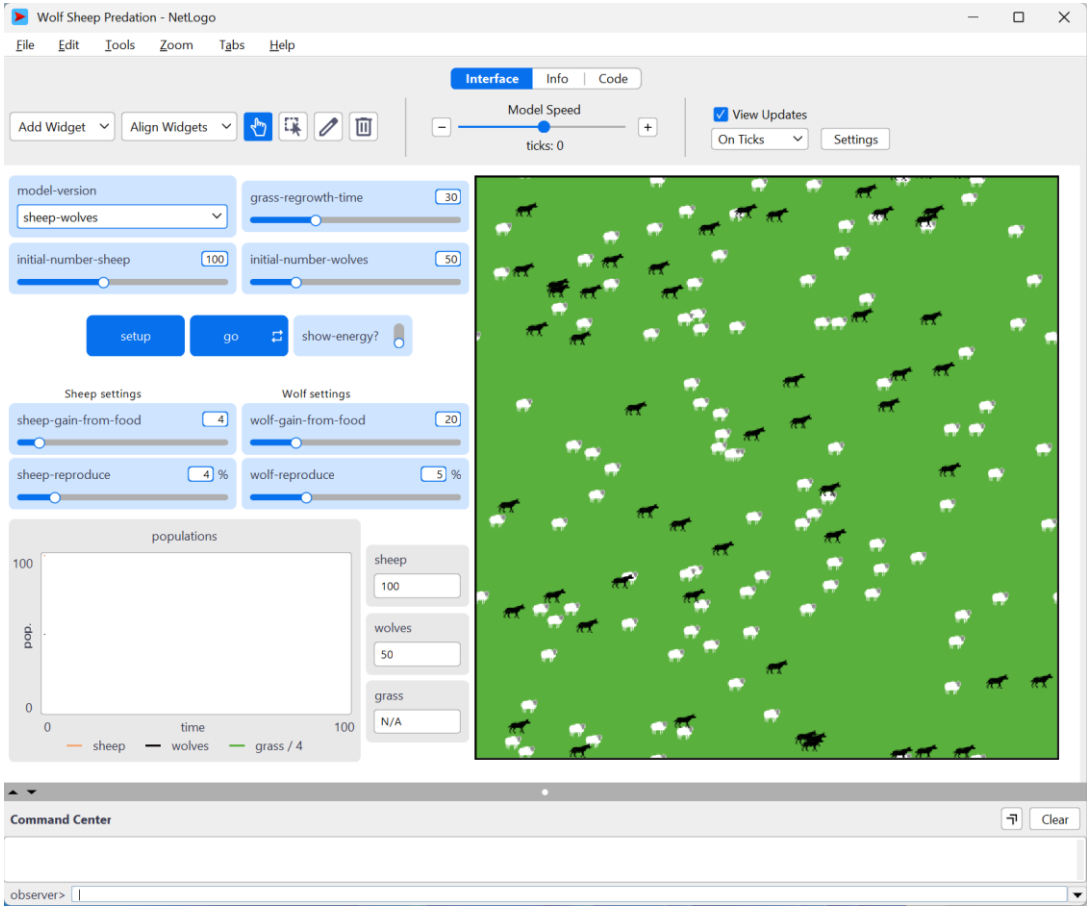
Developing the right intuitions for NetLogo: The Interface

3-Tab structure:

- **Interface** (visualisation, interaction)
- Info (description of the ABM)
- Code (logic of the ABM)

The interface comes with:

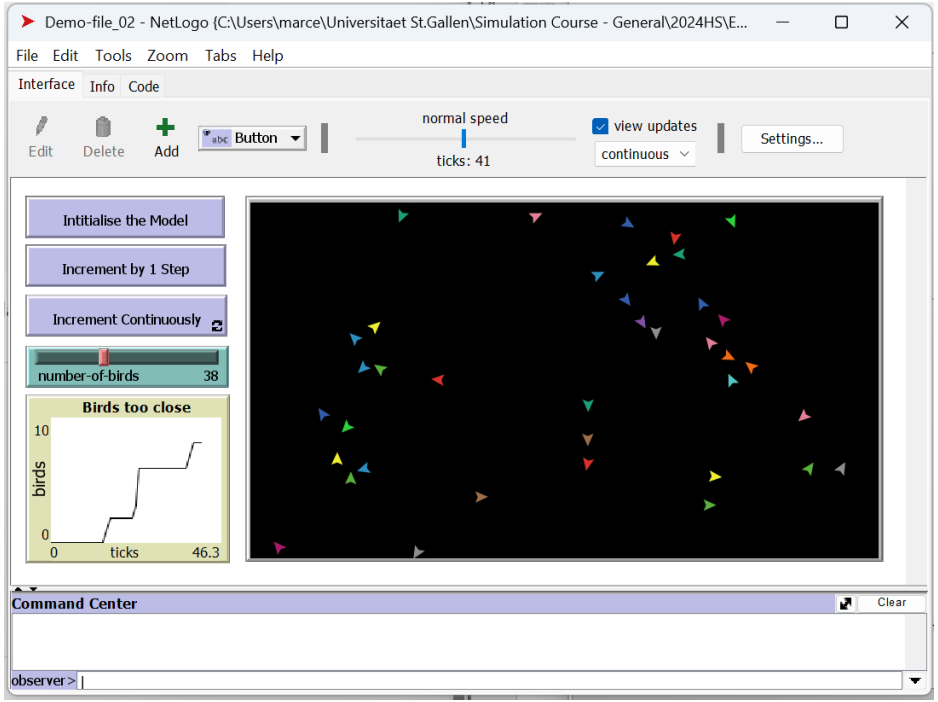
- Buttons (e.g., setup, go)
- Sliders (e.g., #agents, or speed)
- Monitors/Plots (e.g., tracks #agents)
- A visual representation of the state of the simulation (environment)



Demo: How to create the interface?

Let's create an interface with:

- A view of size 16x9
- A button
  - to initialize the model (setup)
  - to increment the state by 1 (go)
  - to continually increment the state by 1 (go)
- A slider to choose the number of birds
- A monitor that indicates the number of birds that fly too close to each other in any given timestep.



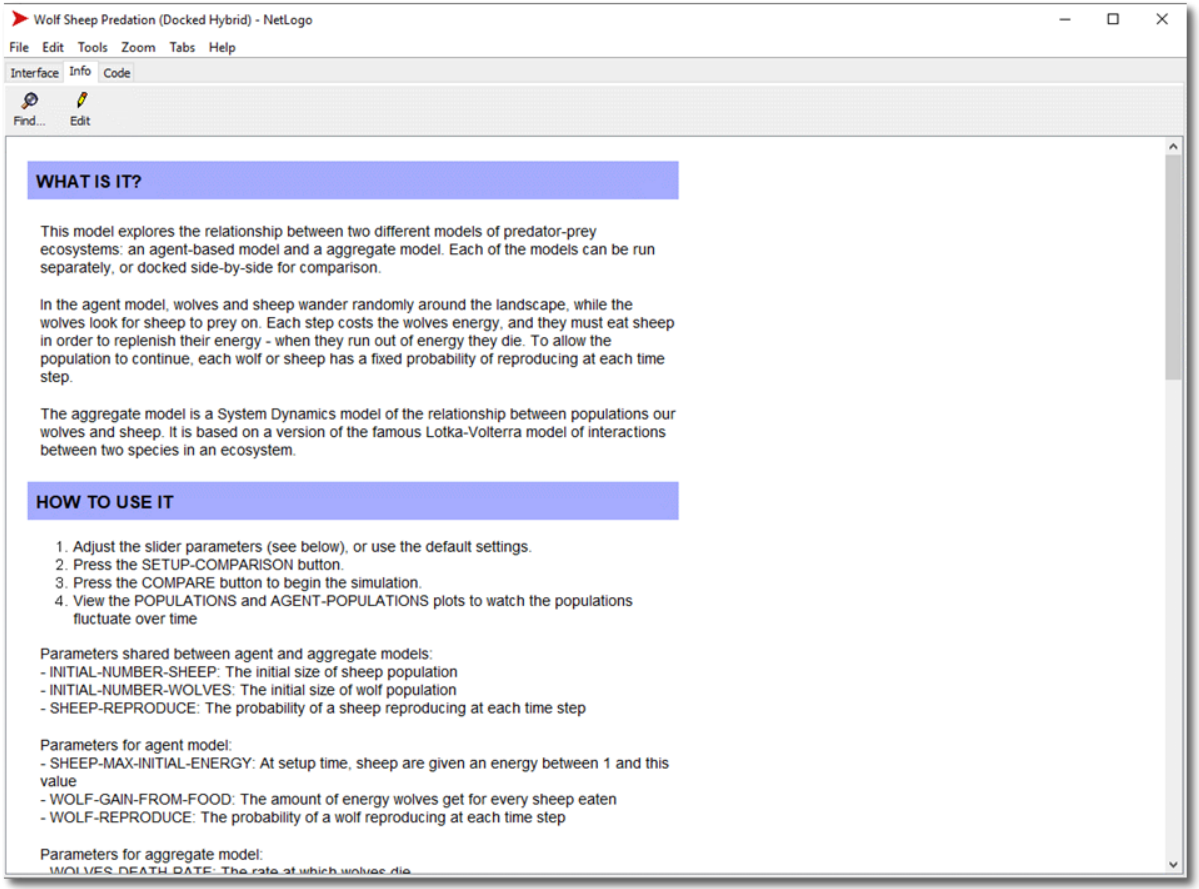
Note: This won't do anything yet, until we touch the code tab—apart from **specifying global variables**.

Developing the right intuitions for NetLogo: Info

3-Tab structure:

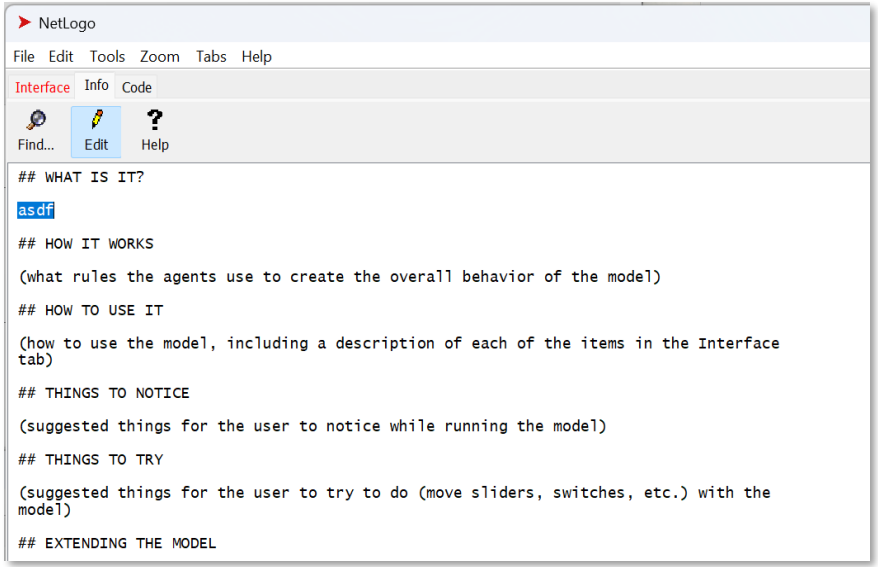
- Interface (visualisation, interaction)
- **Info** (description of the ABM)
- Code (logic of the ABM)

The Info Tab comes with titles (purple) and hints as to what to write in which information section. It helps structure the information necessary for third party users to understand and use the simulation.



Demo: How to add info?

(no explanation needed)



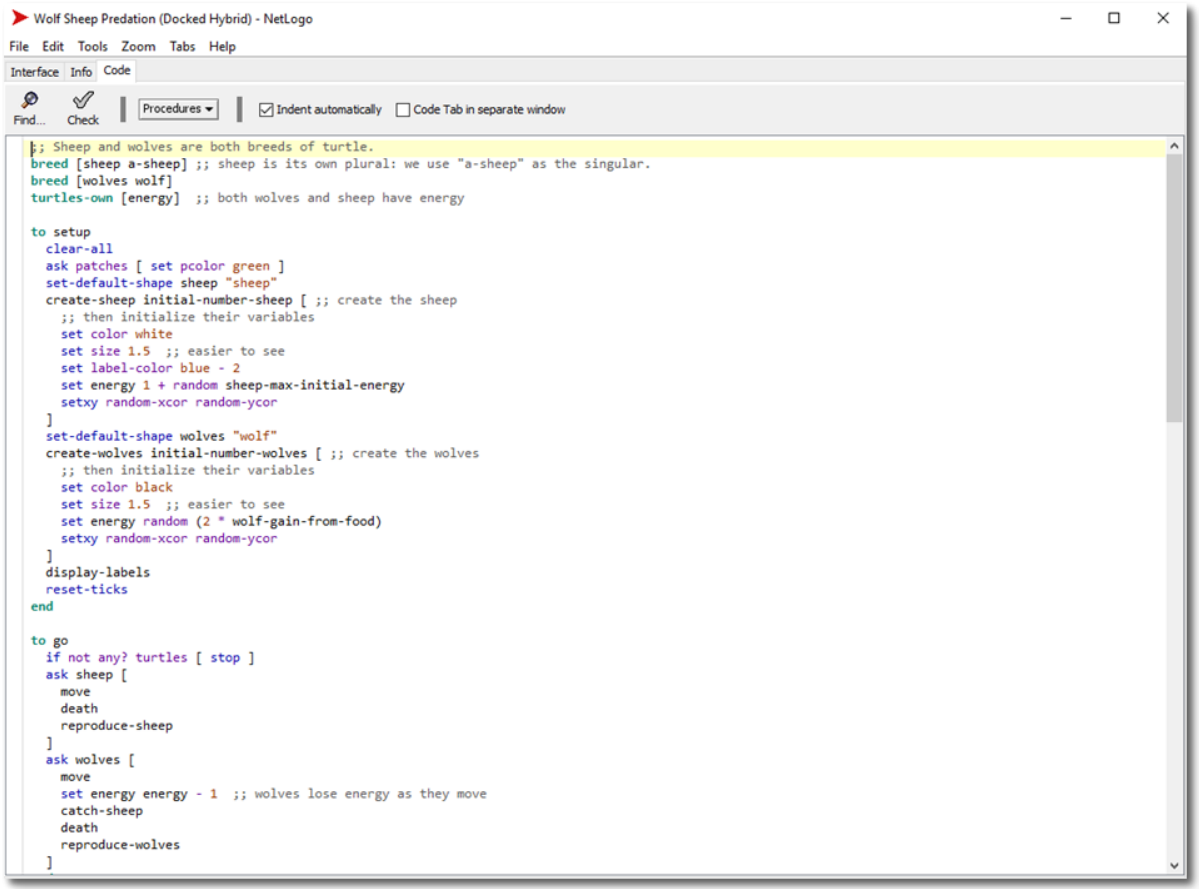
Developing the right intuitions for NetLogo: Code

3-Tab structure:

- Interface (visualisation, interaction)
- Info (description of the ABM)
- **Code** (logic of the ABM)

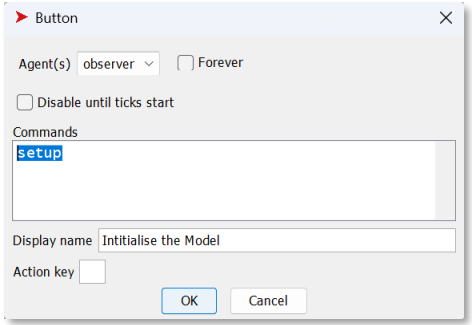
The Code section contains the logic, including:

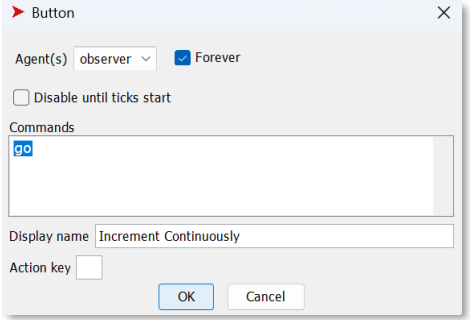
- Setting up the simulation («init»)
- Adoption of changes based on interface interaction (e.g., slider)
- Running of the simulation (i.e., iterating through commands in time intervals and updating states)
- Reporting about the model



Demo: How to code?

A very short demo.





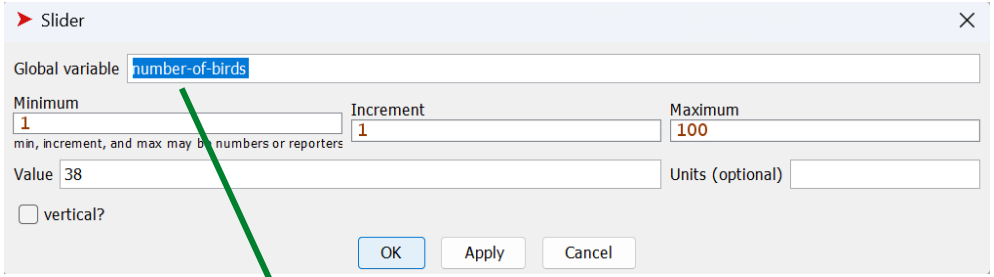

```
globals [min-distance]

to setup
  clear-all ; resets the model/all attribute values
  create-turtles number-of-birds [
    setxy random-xcor random-ycor
    set size 4
  ]
  reset-ticks ; we need this to track bird crashes over time
end

to go
  ask turtles [
    forward 1 ; move by 1 step
    if any? other turtles-here [ set min-distance min-distance + 1 ]
  ]
  tick ; increment the time steps by 1
end
```

Demo: How to code?

A very short demo.



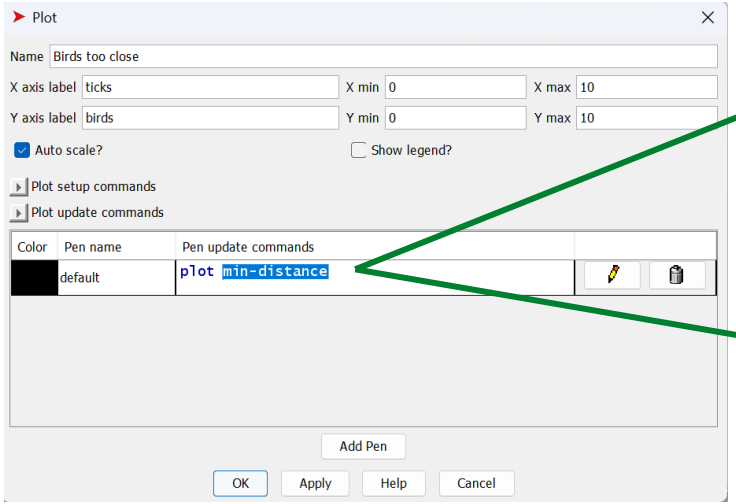

```
globals [min-distance]

to setup
  clear-all ; resets the model/all attribute values
  create-turtles number-of-birds [
    setxy random-xcor random-ycor
    set size 4
  ]
  reset-ticks ; we need this to track bird crashes over time
end

to go
  ask turtles [
    forward 1 ; move by 1 step
    if any? other turtles-here [ set min-distance min-distance + 1 ]
  ]
  tick ; increment the time steps by 1
end
```

Demo: How to code?

A very short demo.




```
globals [min-distance]

to setup
  clear-all ; resets the model/all attribute values
  create-turtles number-of-birds [
    setxy random-xcor random-ycor
    set size 4
  ]
  reset-ticks ; we need this to track bird crashes over time
end

to go
  ask turtles [
    forward 1 ; move by 1 step
    if any? other turtles-here [ set min-distance min-distance + 1 ]
  ]
  tick ; increment the time steps by 1
end
```

Agenda

1	Recap of Week 2	4
2	Introduction to NetLogo	8
3	**Coding with NetLogo**	**21**
4	Getting our hands dirty	32

NetLogo: Built-in agent types

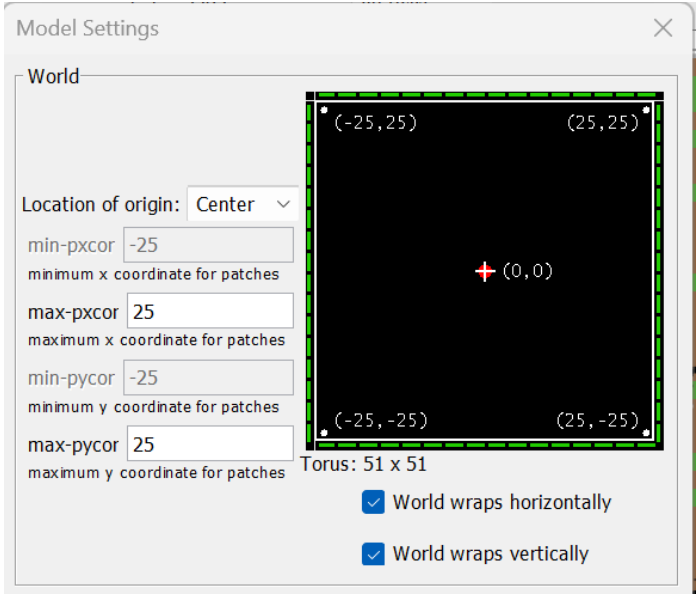
- NetLogo knows 4 types of agents:
  - Turtles (components with agency that move)
  - Patches (stationary agents, tiles of the world)
  - Links (connections between specific turtles)
  - Observer (observes and commands the turtles and patches)

Notice that these are conceptually different types of agents; but when we talked about agent types during the lecture, we talked about different breeds of turtles.

NetLogo: The environment

- The world (environment) consists of tiles with an associated coordinate system.
- By default, the coordinates (0, 0) are in the center.

- Coordinates are used by turtles and patches:
  - Turtles have coordinate attributes (xcor, ycor).
  - Patches have coordinate attributes (pxcor, pycor)
  - xcor/ycor can have decimals, pxcor/pycor cannot
  - Links are ‘lines’ between two turtles (no coord.)



NetLogo: Variables and attributes

- Variables can be global (system-wide values) or specific to turtles, patches, or even links:
  - To define a global variable, use this syntax

```
globals [ myvar anothervar var3 ]
```

  - To define variables associated with something:

```
turtles-own [ speed lineofsight ]
patches-own [ grassgrowthrate ]
links-own [ linkstrength ]
```

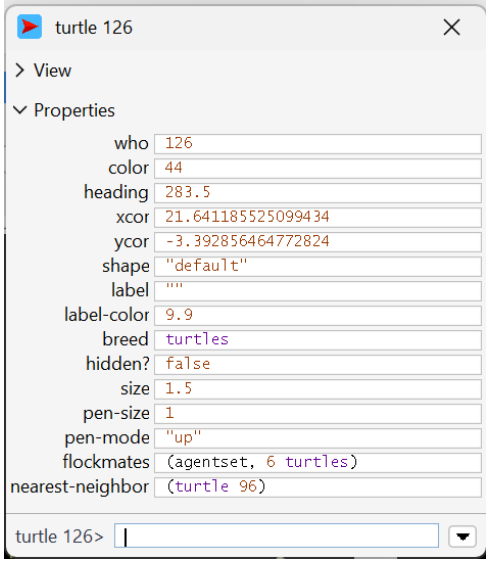
- You can always access/set/change global variables.
- A specific turtle can access/set/change the variables of the patches it is standing on.
- Use the 'set' primitive to change values (see below)

```
turtles-own [
  flockmates
  nearest-neighbor
]

to setup
  clear-all
  create-turtles population
    [ set color yellow - 2 + random 7
      set size 1.5  ;; easier to see
      setxy random-xcor random-ycor
      set flockmates no-turtles ]
  reset-ticks
end
```

Create attributes that will be associated with each turtle.

Instantiate n=population turtles with these attribute values.



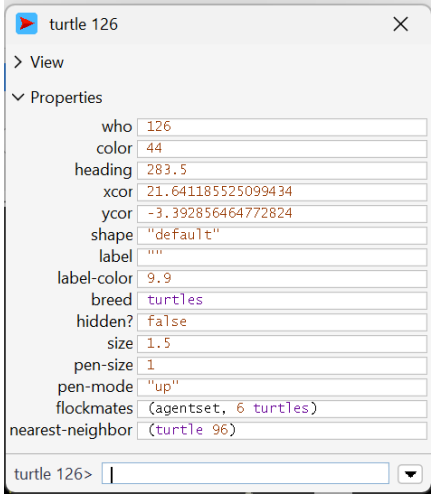
Click on an agent to see its attribute values

NetLogo: Default properties

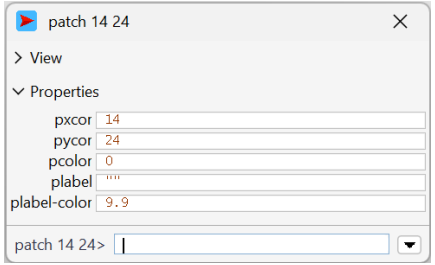
- Agent types (turtles, patches, links) come with default properties. See also built-in variables.
- If you inspect any specific agent, you can see which properties they have (see default properties per agent type to the right).

Applied to your projects:

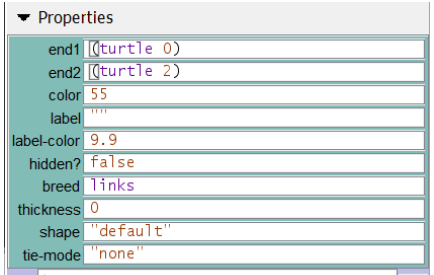
- Use the default properties for your purposes.
- Where needed, create new properties for any of the types (turtles, patches, links), or create sub-types (i.e., breeds) and associate properties with the sub-types only (see: next page).



turtles



patches



links

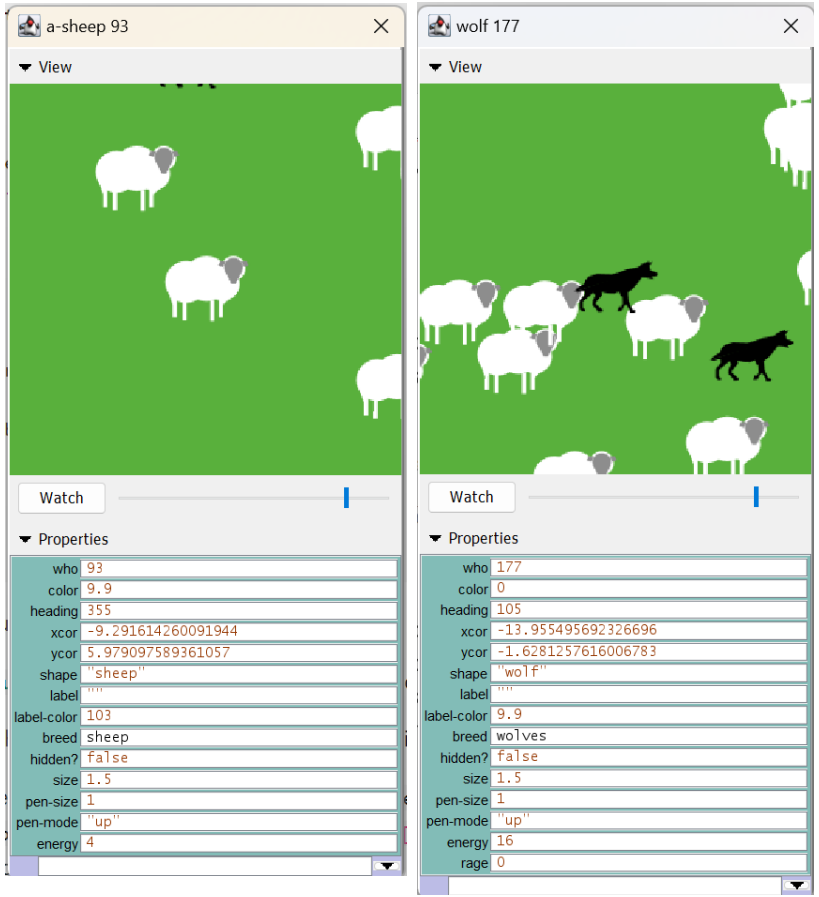
NetLogo: Breeds

- Turtles all share the same list of attributes.
- If you want to change this, you will need to create breeds of turtles, at the top of your code:

```
;; Sheep and wolves are both breeds of turtle.
breed [sheep a-sheep] ;; sheep is plural: we use "a-sheep" as singular
breed [wolves wolf]
turtles-own [energy]  ;; both wolves and sheep have energy
wolves-own [rage] ;; attribute that only applies to the wolves.
```

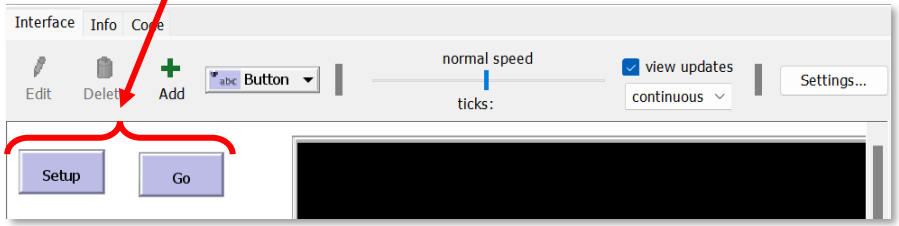
- By using different breeds, you can implement your own agent types, with
  - Type-specific attributes
  - Type-specific behaviours

- Behaviours are implemented with procedures (next page)



NetLogo: Procedures, setup, and go

What should happen, if we pressed on these buttons?
→ A procedure should be triggered.



- Procedures (think 'functions') govern the behaviour
  - To define a procedure, use the to-end-primitive:

```
to procedurename
  your code
end
```

- Essential procedures are 'setup' and 'go'
  - setup: reset the model (clear-all or ca) and create (create-turtles or crt) a number of turtles and the environment
  - go: run the model by calling a number of procedures once or continuously

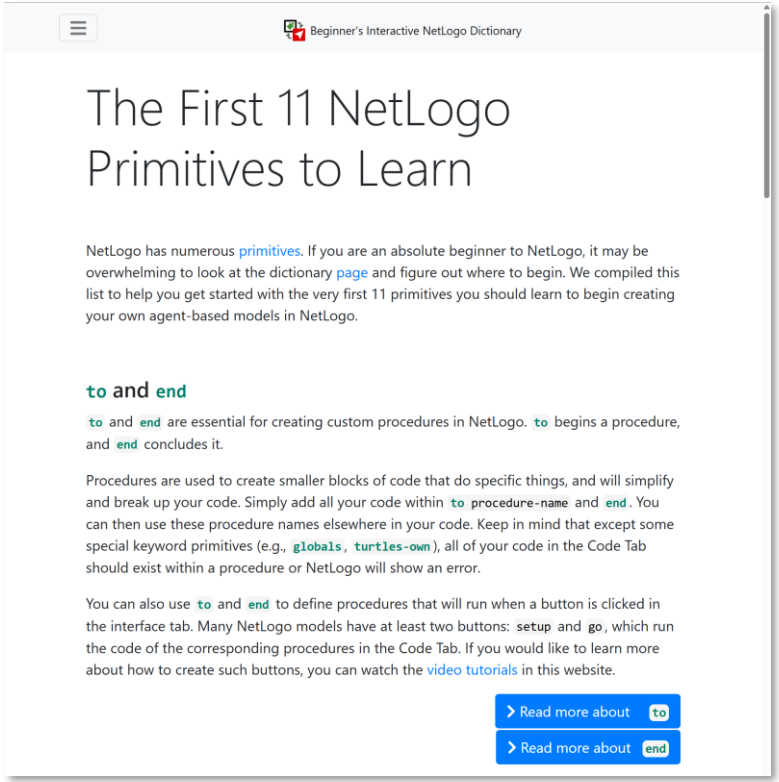
```
to setup
  clear-all     ; you can also write 'ca'
  crt 100       ; creates 100 turtles
end

to go
  ask turtles [     ; to interact with turtles, we 'ask' them
    forward 1       ; move by 1 step
    rt random 45    ; rotate with a random change of angle between 0 and 45
  ]
end
```

NetLogo: Primitives

- NetLogo comes with various ‘primitives’, which are the building blocks of NetLogo programming.
- You can find the documentation here, and a dictionary of the primitives here.

- To start, you may look into these first 11 primitives:
  - to *and* end
  - turtles *and* patches
  - create-turtles
  - ask
  - set
  - forward
  - right
  - if
  - clear-all



Beginner's Interactive NetLogo Dictionary

The First 11 NetLogo Primitives to Learn

NetLogo has numerous primitives. If you are an absolute beginner to NetLogo, it may be overwhelming to look at the dictionary page and figure out where to begin. We compiled this list to help you get started with the very first 11 primitives you should learn to begin creating your own agent-based models in NetLogo.

`to` and `end`

`to` and `end` are essential for creating custom procedures in NetLogo. `to` begins a procedure, and `end` concludes it.

Procedures are used to create smaller blocks of code that do specific things, and will simplify and break up your code. Simply add all your code within `to procedure-name` and `end`. You can then use these procedure names elsewhere in your code. Keep in mind that except some special keyword primitives (e.g., `globals`, `turtles-own`), all of your code in the Code Tab should exist within a procedure or NetLogo will show an error.

You can also use `to` and `end` to define procedures that will run when a button is clicked in the interface tab. Many NetLogo models have at least two buttons: `setup` and `go`, which run the code of the corresponding procedures in the Code Tab. If you would like to learn more about how to create such buttons, you can watch the video tutorials in this website.

Read more about `to`

Read more about `end`

NetLogo: Primitive ‘ask’

- To give instructions to turtles (or patches, or links), we use the primitive ‘ask’ and indicate the target we want to implement the given instructions:

```
to go
  ask turtles [
    forward 1
    rt random 45
  ]
end
```

- To give instructions to a specific breed of turtles, we ask that specific breed, rather than turtles:

```
ask wolves [
  move
  set energy energy - 1  ; wolves lose energy as they move
  eat-sheep ; wolves eat a sheep on their patch
  death ; wolves die if they run out of energy
  reproduce-wolves ; wolves reproduce at a random rate governed by a slider
]
```

NetLogo: Primitive ‘set’

- We can also ask turtles or patches to take on specific **colours**. Notice that we not only need to use the correct syntax for addressing the respective target, but also use the correct colour codes.

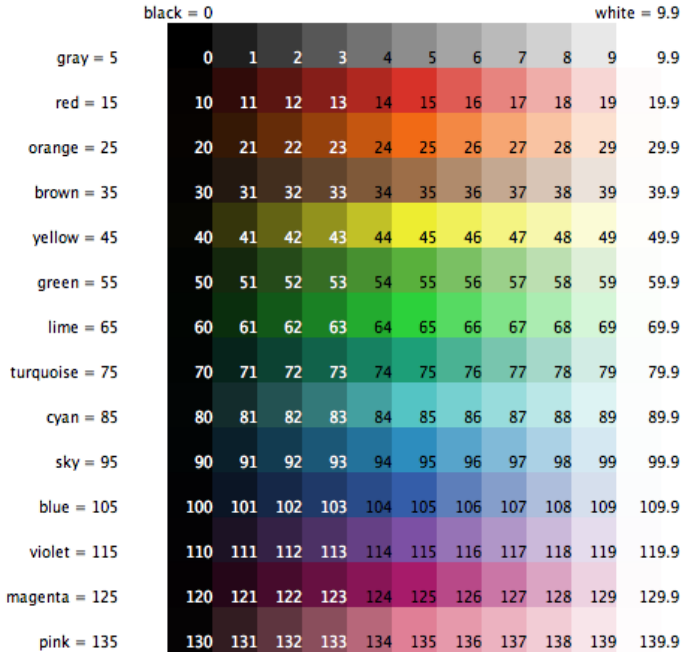
- Keywords/parameter names:

  color colour in association with a turtle
  pcolor colour in association with a patch

- To change the color, we can use the ‘set’ primitive.

```
to setup
  ca
  create-turtles 10 [setxy random-xcor random-ycor]
  colorize
end

to colorize
  ask patches [ set pcolor green ]
  ask turtles [ set color 45 ]
  ask turtles [ set pcolor blue ]
end
```



… or set any other attribute value using the same syntax.

Other useful stuff

Address a selection of objects collectively:

- Built-in collections:
  - turtles, patches, links → addresses all turtles, all patches, and all links respectively
  - turtles-here → addresses all turtles on a given patch
  - in-link-neighbors → addresses the turtles on both ends of a link
- Specific breeds: using breed names → addresses all members of that breed, e.g., ask wolves
- Agentsets:
  - with → addresses agents that have a characteristic based on a condition, e.g., with [speed > 0.5]
  - turtles-on → addresses all turtles on the given patch, patches or on the patch of a given turtle or turtles

To create monitors, plots, or other output data, we can use built-in reporters, agentsets and their combination:

- Agentsets (e.g., max-n-of, max-one-of, any?, count, in-radius, sort-by, with) or reporters (ctrl+f "[reporter]")
- Custom: Use the to-report primitive, to create a new reporter. A reporter returns information.

Agenda

1	Recap of Week 2	4
2	Introduction to NetLogo	8
3	Coding with NetLogo	21
4	**Getting our hands dirty**	**32**

Assignment

1. Re-Create the simulation illustrated on page 14.
   (Do not refer to the solution (p. 18-20), but try to figure it out on your own. Use the linked documentation and explanations on primitives, or this guide's contents (top-right).)

2. Go through the Models Library in NetLogo. Find a model that interests you and analyse it.
   1. Interface: What are the elements used, and how are they implemented?
   2. Code: Find out how the code works by…
      1. Identifying the setup procedure. How is the model initialised?
      2. Identifying the go procedure. What happens in each time step?
      3. Look at the other procedures. What do they do?
   3. Change something in the model. Can you make it work?

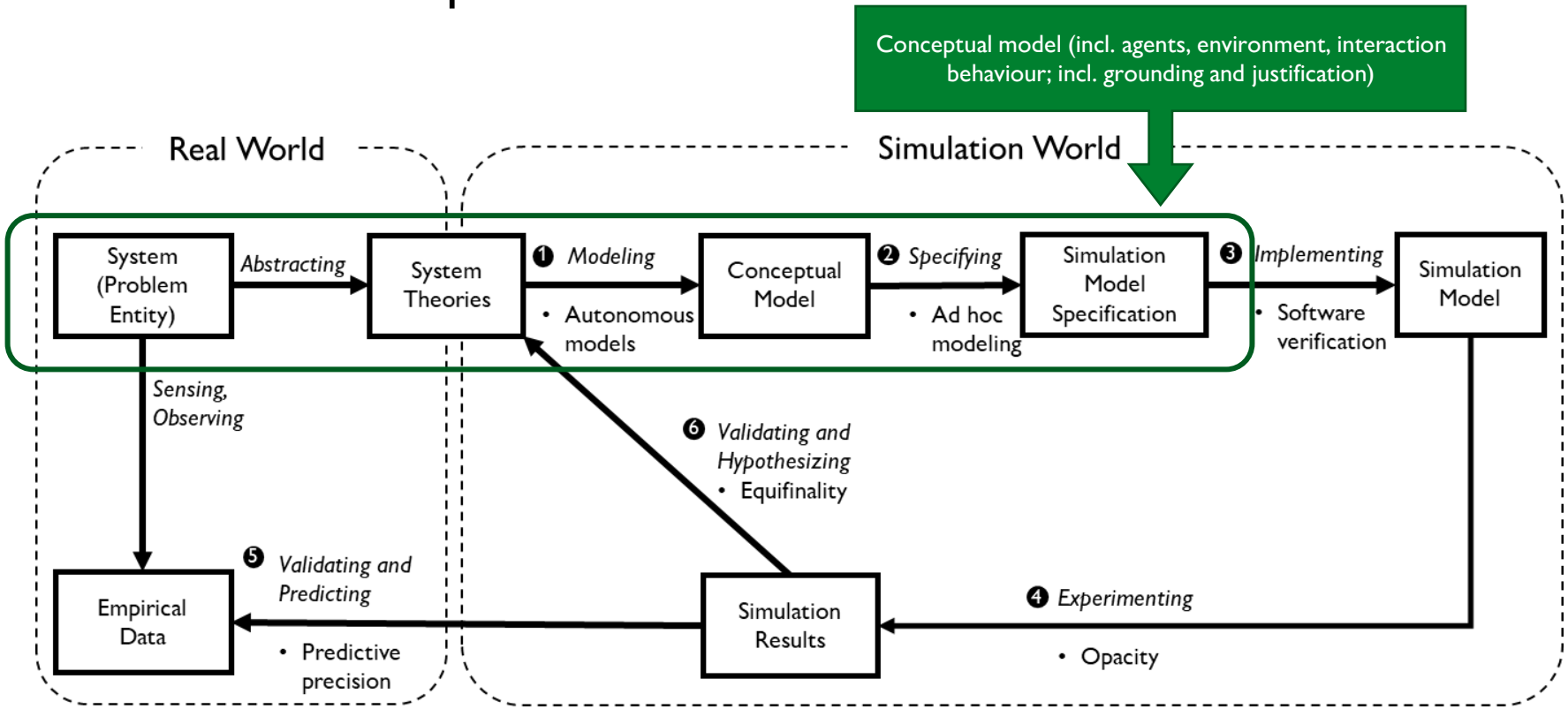
What's next?
Weeks 4-5: Group Work

Week 4: Independent case work (no input session)

Work on your group's model, such that you can present and discuss the following in Week 5:

- Clearly described phenomenon (incl. setting, stakeholders, their goals, courses of action, research questions)
- Conceptual model (incl. agents, environment, interaction behaviour; incl. grounding and justification)
- Your approach to implementing the model (no code expected)
- Expected insights

Week 5: Group Presentations

Presentations per group on the points above. You will have an individual presentation slot for your group, but we encourage you to hear the other group's presentations, too. Slots are communicated on Canvas until Week 4.

Reminder on Computer Simulation